Laplace Smoothing, kNN, Snooping, Noisy Targets and the Curse of Dimensionality

3

Post

CS 3244
Machine Learning

NUS National University of Singapore | Computing

Recap and Forecast

Concept learning as search through $\mathcal{H}$

General-to-specific ordering over $\mathcal{H}$

Candidate elimination algorithm

Boundaries S and G characterize learner's uncertainty

Active learner can generate informative queries

Stronger inductive bias allows classification of greater proportion of unobserved input instances

Topics for this week

- Bayes' rule and Naïve Bayes
- Distance metrics and k Nearest Neighbours
- Training and testing times for both

- Smoothing
- Cost (Loss) Functions
- Noisy Targets
- Curse of Dimensionality

Laplace Smoothing

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Weather Dataset

Outlook	Temperature	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Recap: Likelihood tables for NB

These statistics make up the model representation for NB, our h_θ.

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Temperature	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Windy	True	3/9	3/5
	False	6/9	2/5

Applying NB at test time

Outlook	Temperature	Humidity	Windy	Play Golf
~~Rainy~~ Overcast	Cool	High	True	?

$$P(Yes|x) = P(Overcast|Yes) \times P(Cool|Yes) \times P(High|Yes) \times P(True|Yes) \times P(Yes)$$

$$P(No|x) = P(Overcast|No) \times P(Cool|No) \times P(High|No) \times P(True|No) \times P(No)$$

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Temperature	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Windy	True	3/9	3/5
	False	6/9	2/5

Applying NB at test time

Outlook	Temperature	Humidity	Windy	Play Golf
~~Sunny~~ Overcast	Cool	High	True	?

$$P(Yes|x) = P(Overcast|Yes) \times P(Cool|Yes) \times P(High|Yes) \times P(True|Yes) \times P(Yes)$$

$$P(Yes|x) = \frac{4}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{9}{14} = 0.010582$$

$$\frac{0.0010582}{0.0010582 + 0.0} = 1.0$$

$$P(No|x) = P(Overcast|No) \times P(Cool|No) \times P(High|No) \times P(True|No) \times P(No)$$

$$P(No|x) = \frac{0}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5} \times \frac{5}{14} = 0.0$$

$$\frac{0.0}{0.0010582 + 0.0} = 0.0$$

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Temperature	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Windy	True	3/9	3/5
	False	6/9	2/5

Roll a die 1 time

Lands on 5.

So by maximum likelihood, the next roll will come out as a 5.

$$P(y =' 5' | \theta) = 1$$

But we believe something about dice in general; that dice are fair.

$$P(y = '5') = \frac{1}{6}.$$

That is, we have a *prior* belief.

Photo credits: Wikipedia – Dice

Unseen cases at training

Usually, we don't mean that unseen possibilities at training are improbable to occur.

- x is sampled from the underlying true function f.
- We just might have gotten a sample that doesn't represent such cases.

Want to account for such possibilities through a prior.

Think of saving some probability mass for unseen cases, or "discounting".

Laplace Smoothing

≡ “Add 1 smoothing”

Artificially hallucinate new data.

Add 1 to each possible conditional outcome.

Changes total number of observations for conditionals and priors.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3

Frequency Table

		Play Golf	
		Yes	No
Outlook	Sunny	3+1=4 (4/12)	2+1=3 (3/8)
	Overcast	4+1=5 (5/12)	0+1=1 (1/8)
	Rainy	2+1=3 (3/12)	3+1=4 (4/8)

Priors
$P(\text{Yes}) = 9+1$ (10/16)
$P(\text{No}) = 5+1$ (6/16)

Frequency Table

		Play Golf	
		Yes	No
Outlook	Sunny	3+1=4 (4/12)	2+1=3 (3/8)
	Overcast	4+1=5 (5/12)	0+1=1 (1/8)
	Rainy	2+1=3 (3/12)	3+1=4 (4/8)

Priors
$P(\text{Yes}) = 9+1$ (10/16)
$P(\text{No}) = 5+1$ (6/16)

$$P(Yes|x) = P(Overcast|Yes) \times P(Cool|Yes) \times P(High|Yes) \times P(True|Yes) \times P(Yes)$$

$$P(Yes|x) = \frac{5}{12} \times \frac{4}{12} \times \frac{4}{11} \times \frac{4}{11} \times \frac{10}{16} = 0.01147$$

$$P(No|x) = P(Overcast|No) \times P(Cool|No) \times P(High|No) \times P(True|No) \times P(No)$$

$$P(No|x) = \frac{1}{8} \times \frac{2}{8} \times \frac{5}{7} \times \frac{4}{7} \times \frac{6}{16} = 0.00478$$

Summary – Laplace Smoothing

Overcomes problems with sparse data, unseen observations

"Discounts" some probability mass from observations and assigns to fake

Can apply an arbitrary pseudocount k, rather than 1.

Your turn: Think about how you would set k.
What relationship would we want it to have to $|\mathbf{X}|$?

Many better ways to do this *smoothing* operation

Laplace smoothing implies a uniform prior distribution

K nearest neighbours

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Get to know your neighbors

Let the community decide.

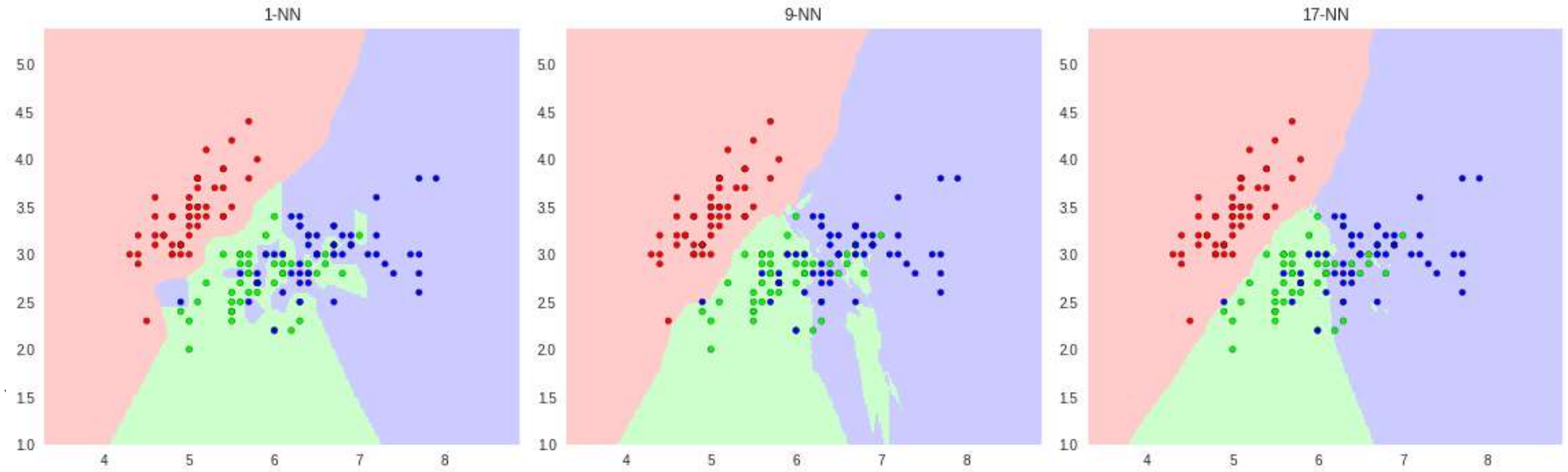
Use k closest training data points, rather than the closest.

Usually set k to an odd number to have fewer ties.

Effect of k

Smaller k: complex surface.

Larger k: smoother surface.



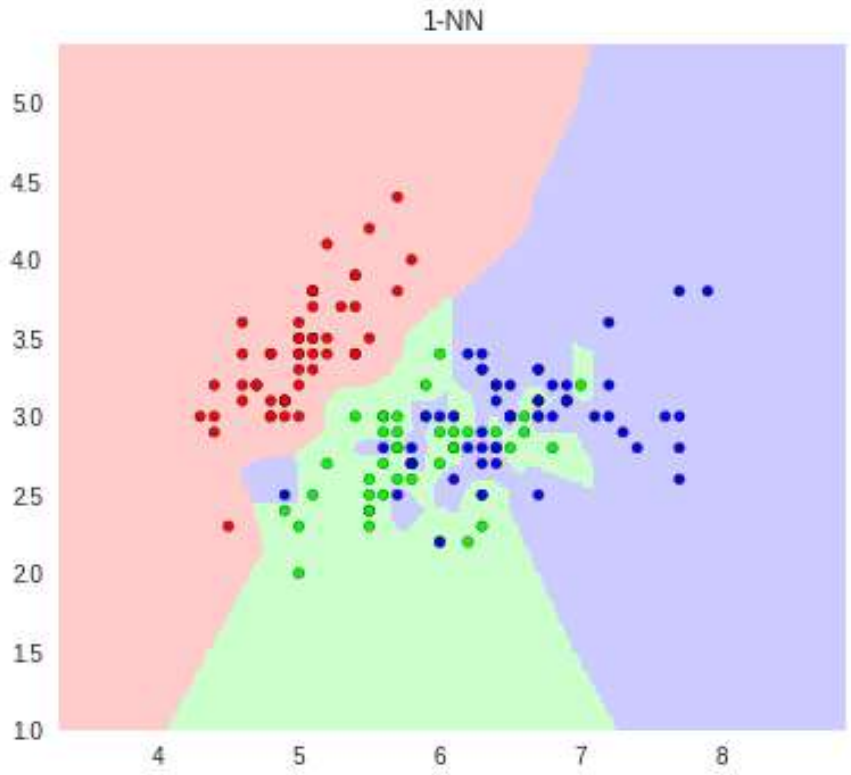
Plot generated by Min @ CS 3244. CC BY 4.0

Perfect performance

What value of k maximizes performance on the training data?

Answer: $k = 1$. Trivially, each point is classified to its own label.

Will it generalize?



Distance Metric

Often we'll use Euclidean distance as our first try:

$$L2\left(x^{(*)}, x^{(i)}\right) = \sqrt{\left(x_1^{(*)} - x_1^{(i)}\right)^2 + \left(x_2^{(*)} - x_2^{(i)}\right)^2 + \cdots + \left(x_n^{(*)} - x_n^{(i)}\right)^2}$$

but any *distance metric* (*non-negativity, symmetric, obeys triangle inequality*) possible.

Defining your distance metric to fit your data's semantic meaning can be important.

Distance Metric

Distilled, here's an example application form that a bank might receive from an applicant. These constitute the *inputs* to the credit approval problem. As above, we can view each instance x as an n dimensional vector (i.e., ordered sequences of n values), where the index within the vector gives the value of a specific criterion: e.g. x_0 yields "32", the applicant's age.

Criterion	Value
Age	32
Gender	Male
Salary (per year)	40K
Debt	26K
Years in Job	1 Year
Years at Current Residence	3 Years

The above input x could thus be represented as $x = [32,\ Male,\ 40K,\ 26K,\ 1\ Year,\ 3\ Years]$. Given an new input x (e.g., a new customer) and a set of historical example inputs with labels, the task in supervised learning is to predict the output label y – should the bank approve or reject the applicant, i.e., $y \in \{approve,\ reject\}$. In supervised learning, a pair of *inputs* and *output* constitute an *instance*: (x, y).

Distance Metrics Behaving Badly

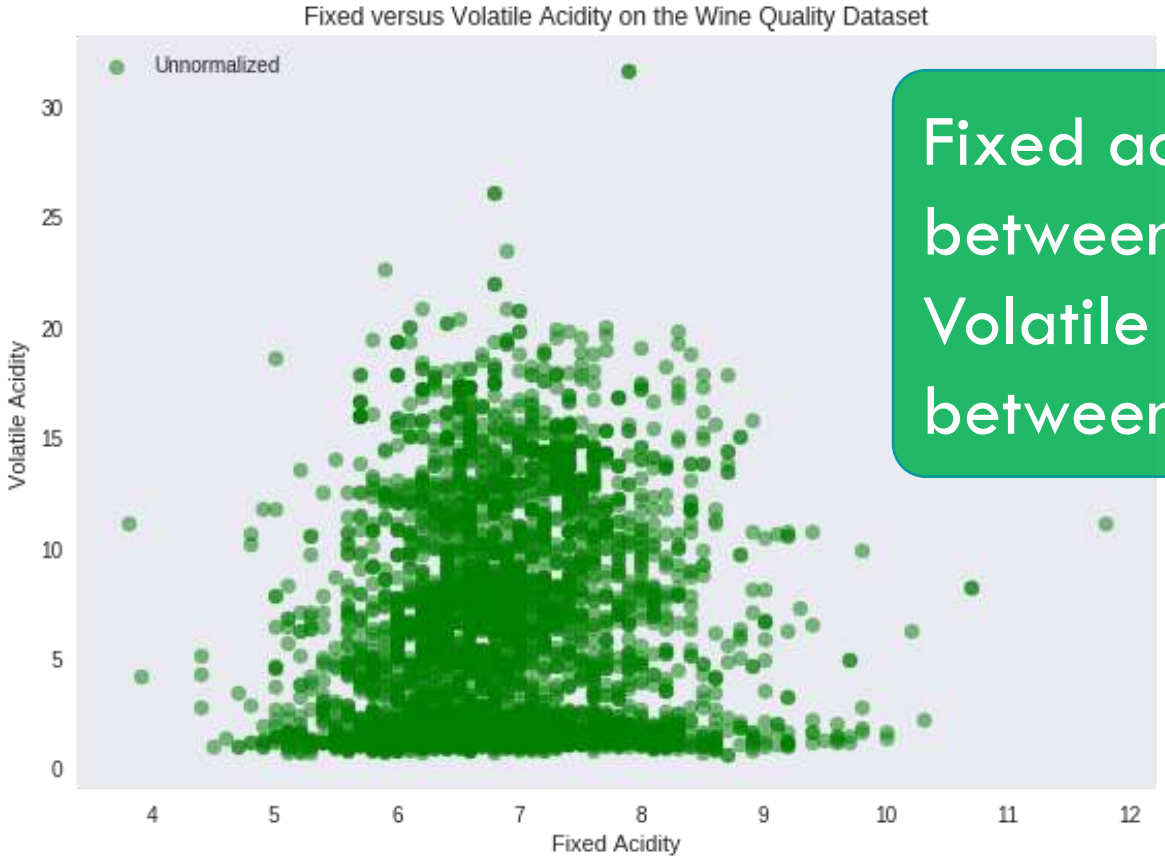
k NN almost never used on images in practice.

Distance metrics on pixels are not informative,
e.g. our L1 distance:

Th

Original image (left) in the public domain via Pexels.

Raw values



Fixed acidity between (-5, 9), Volatile acidity between (0, 15)

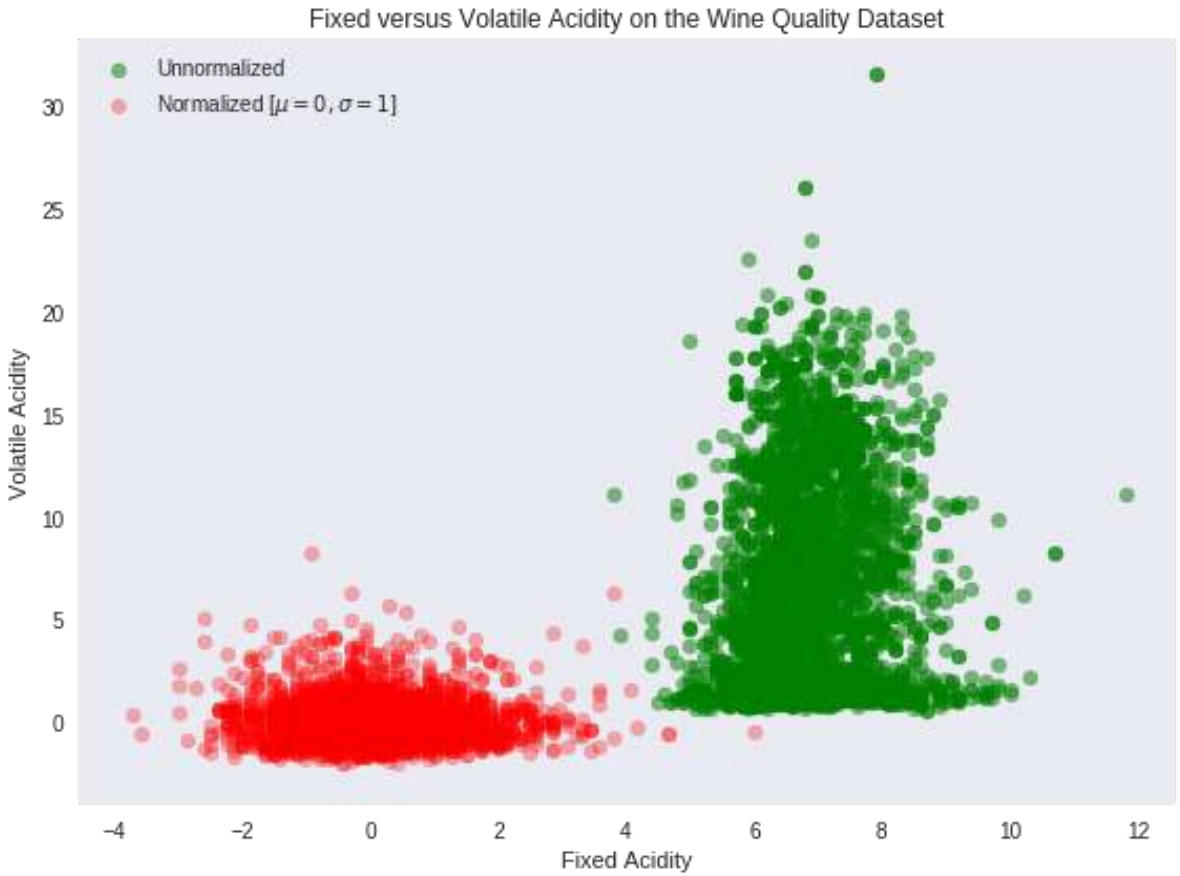
Plot generated by Min @ CS 3244. CC BY 4.0

Raw values

(Standard) Normalization: map $\mu = 0$ and $\sigma = 1$.

Plot generated by Min @ CS 3244. CC BY 4.0

Scaling and Normalization



Plot generated by Min @ CS 3244. CC BY 4.0

Summary – k NN

k Nearest Neighbor predicts a target input's class by the closest k training examples.

Distance metric and k are *hyperparameters*.

Set hyperparameters by:

- Using our prior knowledge of the data
- Using a reserved part of the training data: the *validation* set
- We'll look into this problem in future lectures

Run on the test set once at the end.

Data Snooping

CS3244 Machine Learning

NUS National University of Singapore | Department of Computer Science
School of Computing

Data Snooping

Predict USD versus GBP

Normalize data, split randomly:
$\mathbf{X}_{train}, \mathbf{X}_{test}$

Train only on $\mathbf{X}_{train}$,
Test h_θ on $\mathbf{X}_{test}$

Got great performance!
Let's invest!

$\Delta r_{-20}, \Delta r_{-19}, \ldots, \Delta r_{-1} \rightarrow \Delta r_0$

Chart credits: Monaneko @ Wikimedia Commons

Data Snooping

Lost our $$$. Why?

Normalize data, split randomly:
$\mathbf{X}_{train}, \mathbf{X}_{test}$

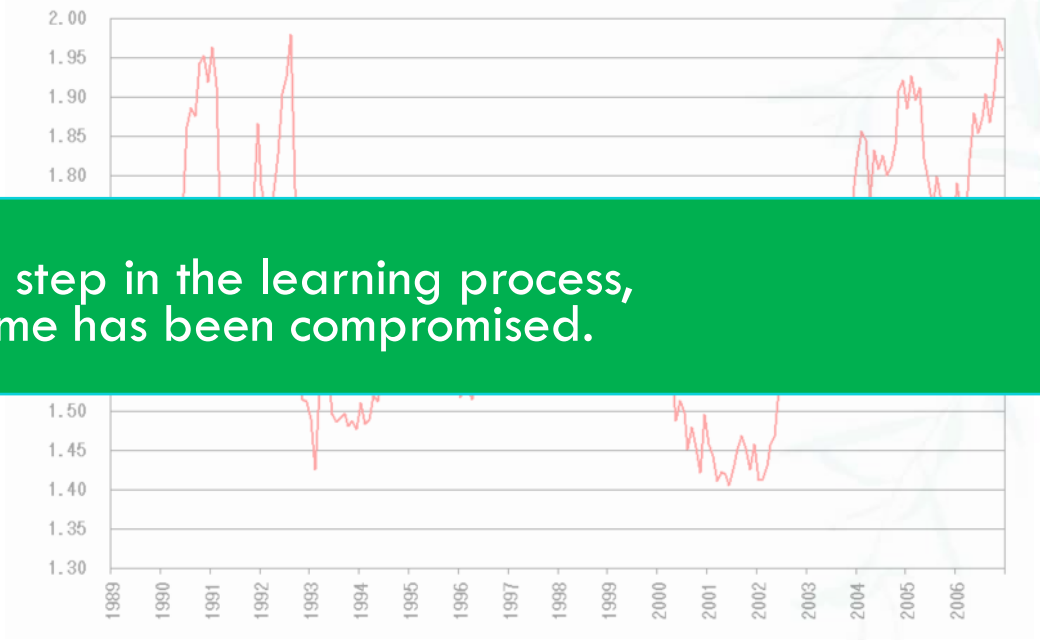
Train only on $\mathbf{X}_{train}$,
Test h_θ on $\mathbf{X}_{test}$

$\Delta r_{-20}, \Delta r_{-19}, \ldots, \Delta r_{-1} \rightarrow \Delta r_0$

Chart credits: Monaneko @ Wikimedia Commons

Data Snooping

Lost our $$$. Why?



If a data set has affected any step in the learning process, its ability to assess the outcome has been compromised.

Most common trap for practitioners – many ways to slip.

Chart credits: Monaneko @ Wikimedia Commons

Noisy Targets

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Noisy targets

The target function isn't always a function $f: \mathcal{X} \to \mathcal{Y}$

Criterion	Value
Age	32 years
Gender	Male
Salary	40 K
Debt	26 K
…	…
Years in Job	1 year
Years at Current Residence	3 years

Consider two identical customers for loan approval …

could have two different outcomes!

Why? And how to we characterize these sources of noise?

Is misreporting salary also a cause of noisy targets?

Target distribution

Instead of saying the target is a function, think of it as a distribution: $P(y|x)$

Our data (x, y) is now generated by the joint distribution:

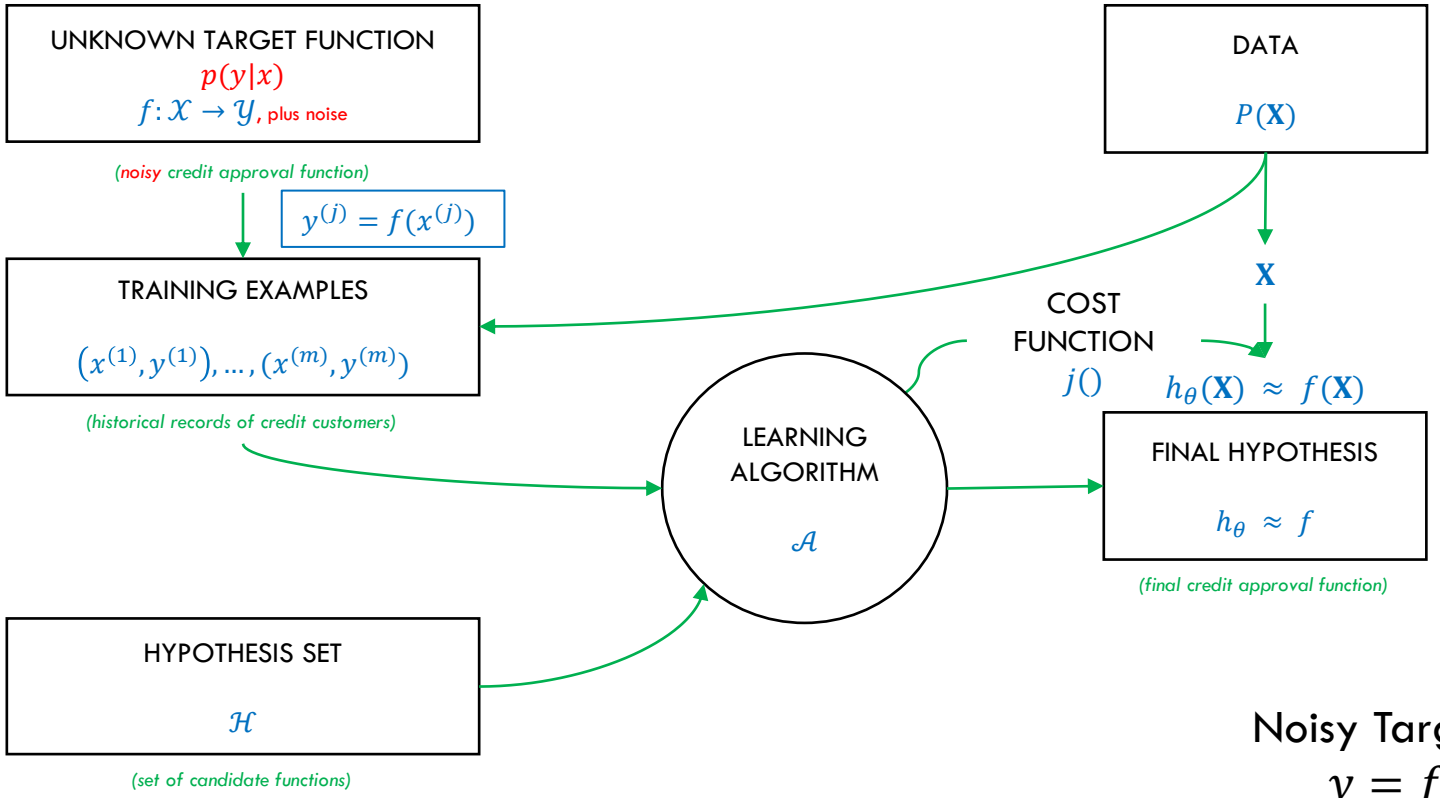
$$P(x)P(y|x)$$

Noisy target $=$ deterministic target function $f(x) = \mathbb{E}(y|x)$
$+$ noise $(y - f(x))$

A deterministic target is just a special case:

$$P(y|x) = 0, \text{ except for } y = f(x)$$

Summary – the learning diagram with noisy targets



Noisy Target:

$$y = f(x) \quad \to \quad y \sim P(y|x)$$

Curse of Dimensionality

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

k NN also suffers from the curse of dimensionality

In high dimensions:

- The number of points needed to keep the same density grows exponentially.
- The distance between points is similar, making it hard to distinguish.

Sparsity with high dimensions

$m = 5$

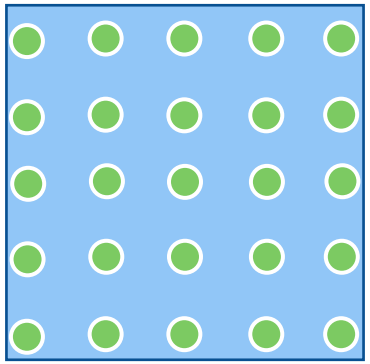
$n = 1$

Chart illustration idea: Stanford CS229N

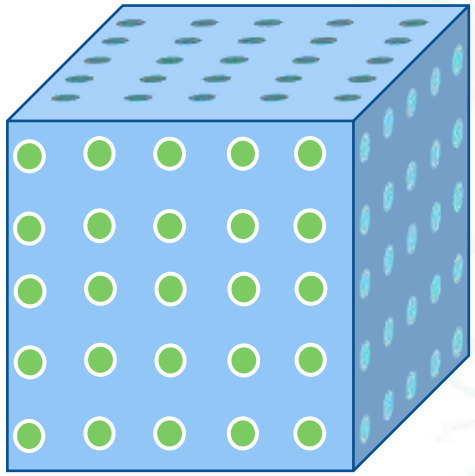
Sparsity with high dimensions

$m = 5$
$n = 1$

$m = 25$
$n = 2$



$m = 125$
$n = 3$



Sparsity: maintaining density of samples depends on exponential growth of the data

Chart illustration idea: Stanford CS229N

Case study: unit hypercube

File Edit View Insert Runtime Tools Help

CODE TEXT CELL CELL CONNECTED

5 Curse of Dimensionality

(Optional) Finally we include code for you to run and modify to check the *Curse of Dimensionality*. Recall that the Curse of Dimensionality suggests that the larger the dimensional space you have in your dataset, the more difficult certain aspects of learning can become.

```
import numpy as np

# Try some on your own
for dim in [1,2,3,4,8,10,100,200]:
    ## choices of |X|
    samples = 200
    #samples = pow(2,dim)
    X = np.random.uniform(size=(samples,dim))

    ## choices of x
    x = np.random.uniform(size=(dim))
    # x = np.zeros(dim)
    print("%d dimensions:" % dim)
    print("#   Generated %d points" % samples)
    # print("#   Test point: ", x)
    dist_array = []
    for j in range(len(X)):
        dist = 0.0
        for i in range(dim):
#           print ("%f => %f "% (x[i]-X[j][i],pow(x[i]-X[j][i],2)))
            dist += pow(x[i]-X[j][i],2)
#    print ("Sum distance %f => %f" % (dist,pow(dist,0.5)))
        dist = pow(dist,0.5)
        dist_array.append(dist)
#    print ("%d th point %s has distance %f" % (j,X[j],dist))
    distances = np.array(dist_array)
    print ("#   Average distance (STD) of test point to %d samples: %f (%f)" % (samples,distances.mean(),distances.std()))
```

σ only grows slowly

```
1 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 0.908554 (0.627194)
2 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 1.357414 (0.672876)
3 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 2.104206 (0.862900)
4 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 2.857518 (0.851753)
8 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 4.298752 (0.915833)
10 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 4.240121 (0.873999)
100 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 14.180229 (0.844941)
200 dimensions:
#   Generated 200 points
Average distance (STD) of test point to 200 samples: 19.588471 (0.845534)
```

Summary – high dimensions

- Hard to visualize
- Difficult to get sufficient samples
- Most points end up far away; Euclidean distance becomes less meaningful
  - High dimensional hypercube is most all “edge” space, far from the center of the hypercube
- Choose or design an appropriate distance metric